

MODULE 5

Computer Animation

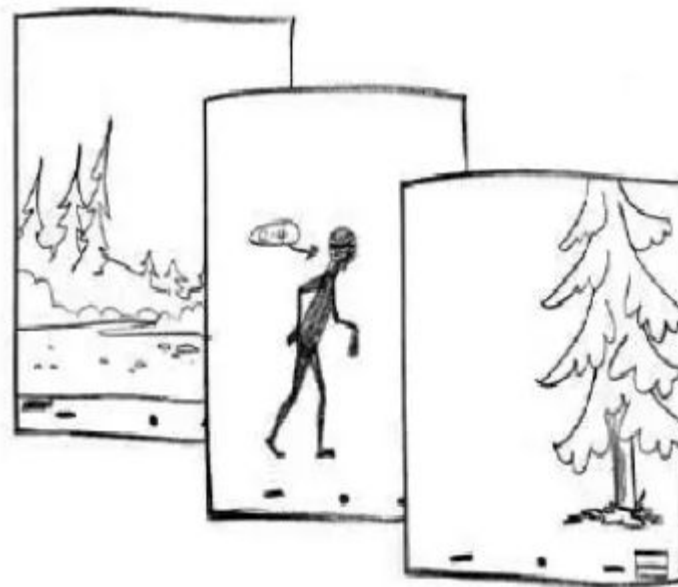
DESIGN OF ANIMATION SEQUENCES

- Storyboard layout
- Object definitions
- Key-frame specification
- Generation of in-between frame

For *frame-by-frame* animation, each frame of the scene is separately generated and stored. Later, the frames can be recoded on film or they can be consecutively displayed in "real-time playback" mode.

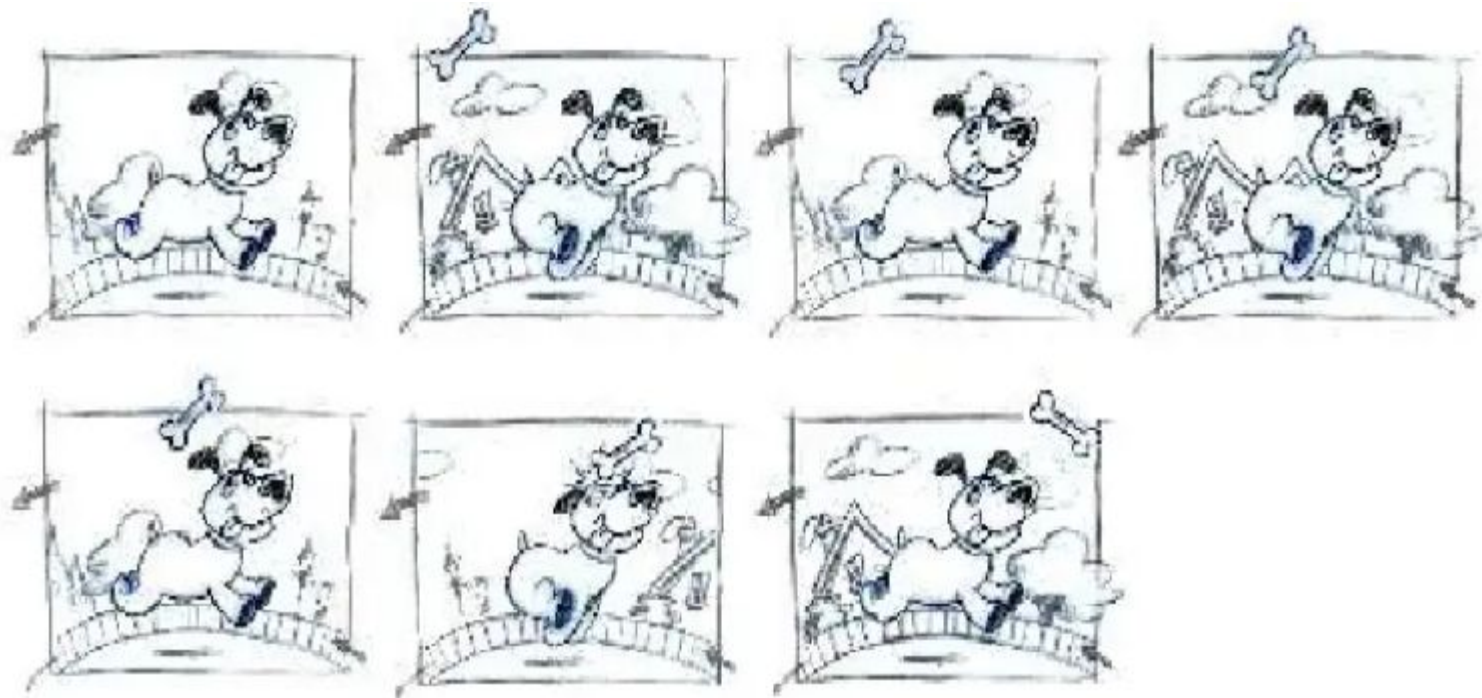
1. Storyboard layout

- A storyboard is a visual representation of how the story depicted in the motion or animation will be told.
- A storyboard is a physical or digital visual overview of a project.
- The storyboard is an outline of the action. It defines the motion sequence as a set of basic events that are to take place.
- Depending on the type of animation to be produced, the storyboard could consist of a set of rough sketches or it could be a list of the basic ideas for the motion.



2. Object definitions

- An object definition is given for each participant in the action.
- Objects can be defined in terms of basic shapes, such as polygons or splines.
- What are the properties that is the color or size of the object.
- In addition, the associated movements for each object are specified along with the shape



3. Key frame specification

- A key frame is a detailed drawing of the scene at a certain time in the animation sequence.
- Within each key frame, each object is positioned according to the time for that frame.
- It is the starting and ending frame point of any animation and film making drawing for smooth transaction.
- It contains sequence of key frame which define movement for viewer eyes.
- 2 or 3 delta frames can be present for a span of a second between frames.
- More key frames are specified for intricate motions than for simple, slowly varying motions.



4. Generation of in-between frame

- In-between are the intermediate frames between the key frames.
- The number of in-betweens needed is determined by the media to be used to display the animation.
- Film requires 24 frames per second, and graphics terminals are refreshed at the rate of 30 to 60 frames per second
- Typically, time intervals for the motion are set up so that there are from three to five in-betweens for each pair of key frames.
- Depending on the speed specified for the motion, some key frames can be duplicated.

- There are several other tasks that may be required, depending on the application.
- They include motion verification, editing, and production and synchronization of a soundtrack

Example

1. Storyboard layout.

- a. In our animation one apple falls from a tree and bounces to the ground.
- b. Then a bird flies in and swirls around in the air to see if any danger is there.
- c. Then it sits on the apple picks it up and flies away.

2. Object Definition

1. A tree:



2. An apple:



3. Mud on the ground:



4. Bird:



Key frames:

1.



2.



▣ 3.



5.







GENERAL COMPUTER-ANIMATION FUNCTIONS

- For designing the animation and providing processing to the individual object, special functions are needed.
- These special functions in animation are present in animation package.
- For development of animation sequence following steps are included.
 - Object manipulations and rendering
 - Camera motions
 - The generation of in-between frames

- Animation packages have on function available which store and manage the object Database.
- Object database object shape & its associated parameters are updated & stored in database.
- Other object animation functions includes
 - Object motion generation
 - Motions can be generated according to specified constraints using two-dimensional or three-dimensional transformations.
 - Object rendering
 - Standard functions can then be applied to identify visible surfaces and apply the rendering algorithms.

- One of the function is to capture or see the movement of camera.
 - Zooming
 - Panning (rotating vertically or horizontally)
 - Tilting
- Finally, given the specification for the key frames, the in-between can be automatically generated.

Raster animations

- Two types of animation
 - Raster animation(pixel)
 - Smallest addressable, smallest controllable elements on display device.
 - Vector animation(scalable-vertex, edges, nodes, path)

Raster animations conti..

- Animation frames made up of pixels rather than scalable components are referred as “raster animation”.
- Raster animation store images as pixel rather than vector and shows more realistic lighting and color.
- In raster animation computer doesn't have to render each frame in real time.

- Each pixel contains information about each colour, position and its brightness
- We can say that raster images are collection of number of pixels
- Two types of raster animation
 - Traditional
 - Modern
- Traditional
 - Use low detailed animation using sprite sheet
 - Sprite sheet allowed the removal of any object that was not needed but disturbed the animation sequence.

- Modern raster animation
 - Each animation frame is either scanned into a computer.
 - Programming language such as Flash, HTML & Java is used.
 - Scaled to fit any window with pre-defined resolution.
 - Eg: JPG,GIF,PNG,TIF

APPLICATION

- 3D & 2D animation rendering
- Mesh filtering
- Real time game designing & graphics
- Animation movies
- Cartoon animation to make realistic

Advantages

- Provide animation control
- Use less memory
- Flexible & can remove selective object
- Support 3D animation

Disadvantages

- Files are much larger
- Scaling leads to multiplication of each pixel which generate a blocky pixelated image
- Costly hardware device requirement.

COMPUTER-ANIMATION LANGUAGES

- Design and control of animation sequences are handled with a set of animation routines.
- A general-purpose language, such as C, Lisp, Pascal, or FORTRAN, is often used to program the animation functions, but several specialized animation languages have been developed.
- Animation functions include a graphics editor, a key-frame generator, an in-between generator, and standard graphics routines.
- The graphics editor allows us to design and modify object shapes, using spline surfaces, constructive solid-geometry methods, or other representation schemes.

- Animation functions included in animation languages:
- *Animation description*
 - *Scene description.*
 - *Action specification.*
- Key-frame systems
- Parameterized systems
- Scripting systems

Scene description

- This includes the positioning of objects and light sources, defining the photometric parameters (light-source intensities and surface-illumination properties), and setting the camera parameters (position, orientation, and lens characteristics).

Action specification

- This involves the layout of motion paths for the objects and camera.
- And we need the usual graphics routines: viewing and perspective transformations, geometric transformations to generate object movements as a function of accelerations or kinematic path specifications, visible-surface identification, and the surface-rendering operations.

Key-frame systems

- Design for set of animation routines for generation the in between frames and user specified key frames separately.
- Usually, each object in the scene is defined as a set of rigid bodies connected at the joints and with a limited number of degrees of freedom.
- If we also allow base rotations, the robot arm can have a total of 12 degrees of freedom.
- The human body, in comparison, has over 200 degrees of freedom.

Parameterized systems

- Allow object-motion characteristics to be specified as part of the object definitions.
- The adjustable parameters control such object characteristics as degrees of freedom, motion limitations, and allowable shape changes.

Scripting systems

- Object specification define by user input script
- Animation sequence also define by user input script
- Library of various object & motions can be constructed.

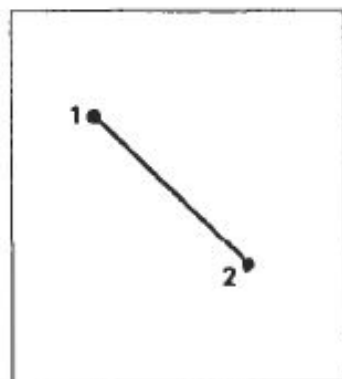
KEY-FRAME SYSTEMS

- We generate each set of in-betweens from the specification of two (or more) keyframes.
- Motion paths can be given with a *kinematic* description as a set of spline curves, or the motions can be *physically based* by specifying the forces acting on the objects to be animated.
- For complex scenes, we can separate the frames into individual components or objects called *cels* (celluloid transparencies),

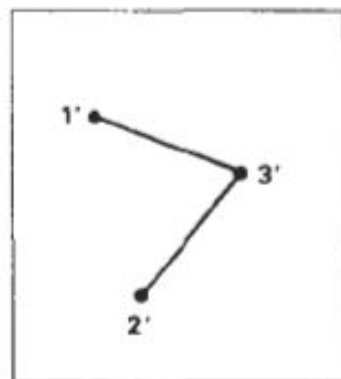
- With complex object transformations, the shapes of objects may change over time.
- If all surfaces are described with polygon meshes, then the number of edges per polygon can change from one frame to the next.
- Thus, the total number of line segments can be different in different frames.

Morphing

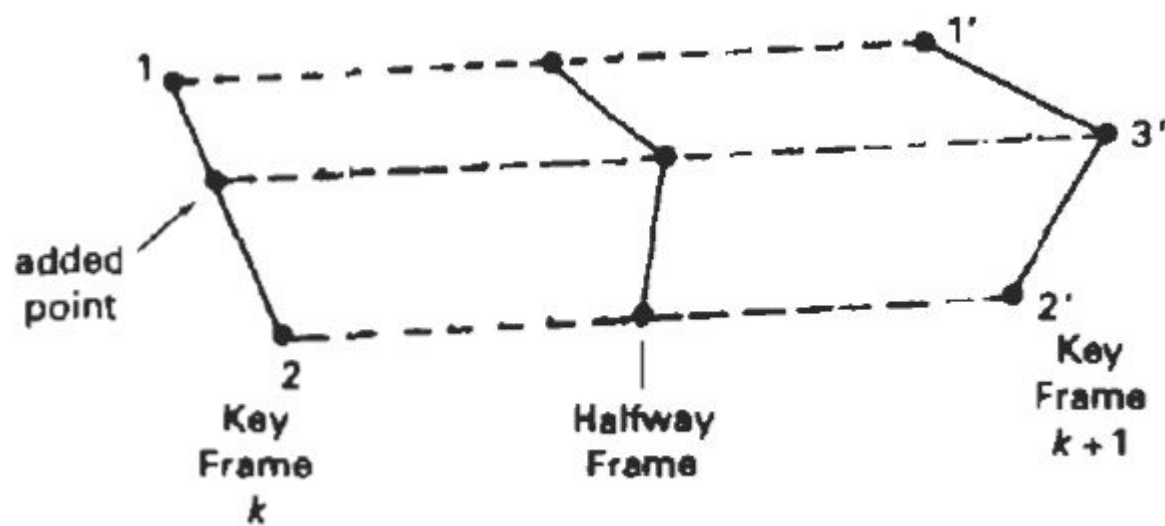
- Transformation of object shapes from one form to another is called morphing,
- Morphing methods can be applied to any motion or transition involving a change in shape.



Key
Frame k

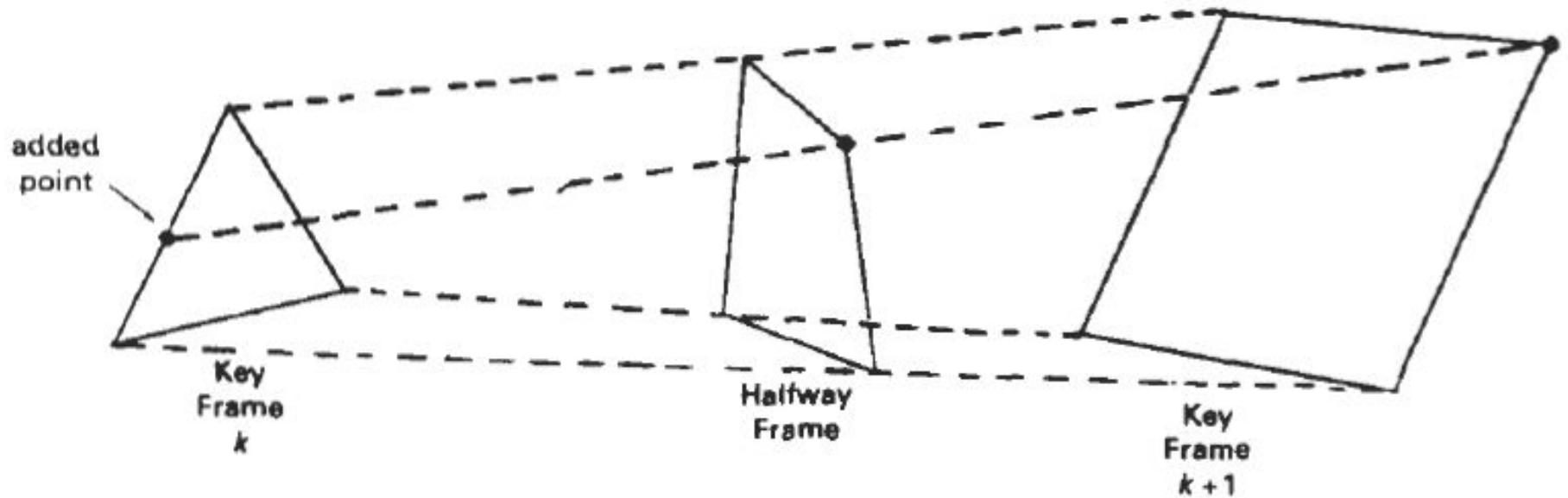


Key
Frame $k+1$



- A straight-line segment in key frame transformed into two line segments in key frame $k+1$.
- Since key frame $k + 1$ has an extra vertex, we add a vertex between vertices 1 and 2 in key frame k to balance the number of vertices (and edges) in the two key frames.
- Using **linear interpolation** to generate the in-betweens.
- We transition the added vertex in key frame k into vertex $3'$ along the straight-line path

An example of a triangle linearly expanding into quadrilateral



- We can state general preprocessing rules for equalizing key frames in terms of either the number of edges or the number of vertices to be added b a key frame.
- Morphing using linear interpolation
 - Edge: let L_K and L_{k+1} denote line segments of two consecutive frames. K initial key frame and $k+1$ is next key frame.

$$L_{\max} = \max(L_k, L_{k+1}), \quad L_{\min} = \min(L_k, L_{k+1})$$

$$N_e = L_{\max} \bmod L_{\min}$$

$$N_s = \text{int}\left(\frac{L_{\max}}{L_{\min}}\right)$$

N_e =Number of edges

N_s =Number of sections

- Then the preprocessing is accomplished by
 1. dividing N_e , edges of $keyframe_{min}$, into $N_s + 1$ sections
 2. dividing the remaining lines of $keyframe_{min}$, into N_s , sections
- If $L_k = 15$ $L_{k+1} = 11$, We divide 4 lines of $keyframe_{k+1}$ into 2 sections each. The remaining lines of $keyframe_{k+1}$ are left intact.

- Vertex: Let V_k and V_{k+1} denote the number of vertices in the two consecutive frames.

$$V_{\max} = \max(V_k, V_{k+1})$$

$$V_{\min} = \min(V_k, V_{k+1})$$

$$N_{ls} = (V_{\max} - 1) \bmod (V_{\min} - 1)$$

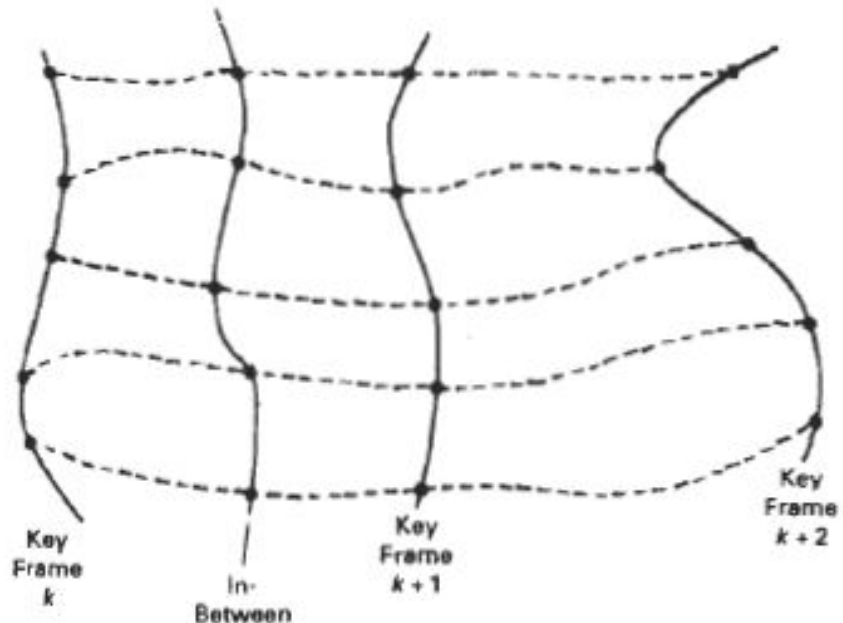
$$N_p = \text{int}(V_{\max} - 1) / (V_{\min} - 1)$$

- Where N_{ls} is number of line sections and N_p number of point.

- Add N_P points to N_{ls} line sections of $\text{keyframe}_{\min}$
- Add $N_P - 1$ points to the remaining edges of $\text{keyframe}_{\min}$
- If $V_K = 3$ and $V_{K+1} = 4$, $N_{ls} = N_p = 1$ So add one point to one edge of keyframe_k . No points would be added to remaining edges of keyframe_k .

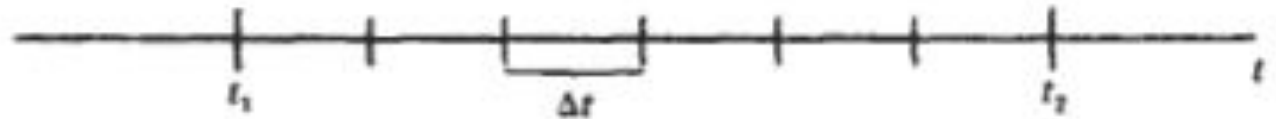
Simulating Accelerations

- Curve-fitting techniques are often used to specify the animation paths between key frames.
- Given the vertex positions at the key frames, we can fit the positions with linear or nonlinear paths.



Fitting key-frame vertex positions with nonlinear splines.

- This determines the trajectories for the in-betweens.
- To simulate accelerations, we can adjust the time spacing for the in-betweens.
- For constant speed (**zero acceleration**), we use equal-interval time spacing for the in-betweens.
- Suppose we want n in-betweens for key frames at times t_1 , and t_2



- The time interval between key frames is then divided into $n + 1$ subintervals, yielding an in-between spacing of

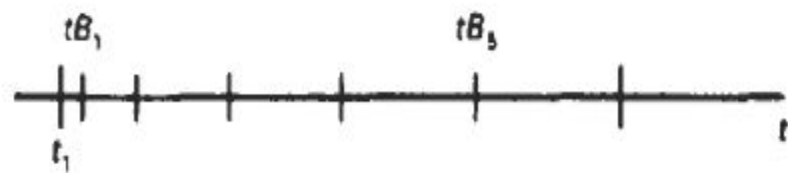
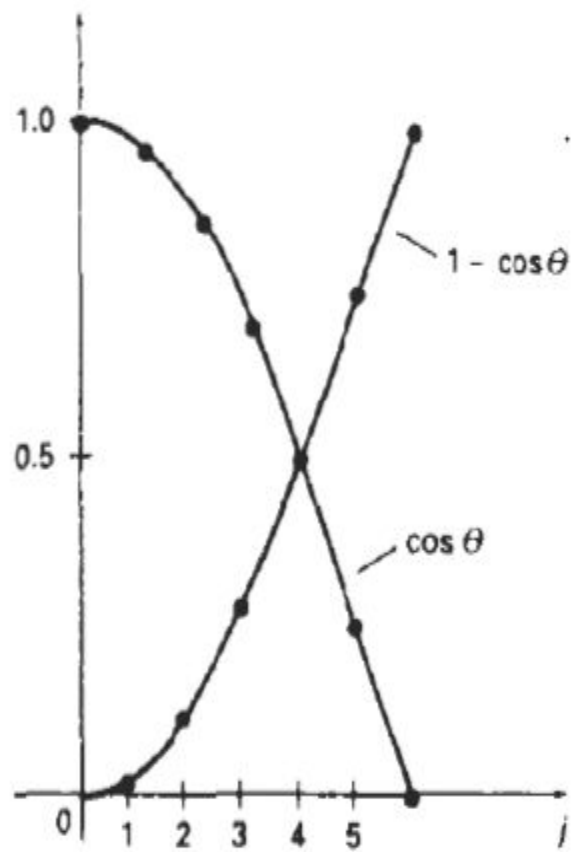
$$\Delta t = \frac{t_2 - t_1}{n + 1}$$

- We can calculate the time for any in-between as

$$t_{Bj} = t_1 + j\Delta t, j=1,2,\dots,n$$

- and determine the values for coordinate positions, color, and other physical parameters.

- **Nonzero accelerations** are used to produce realistic displays of speed changes, particularly at the beginning and end of a motion sequence.
- We can model the start-up and slowdown portions of an animation path with spline or trigonometric functions.
- Parabolic and cubic time functions has been applied to acceleration modeling, but trigonometric functions are more commonly used in animation packages



- To model increasing speed (**positive acceleration**), we want the time spacing between frames to increase so that greater changes in position occur as the object moves faster.
- We can obtain an increasing interval size with the function

$$1 - \cos \theta \quad 0 < \theta < \frac{\Pi}{2}$$

- For n in-betweens, the time for the **jth** in-between would then be calculated as

$$tB_j = t_1 + \Delta t \left[1 - \cos \frac{j \Pi}{2(n+1)} \right]$$

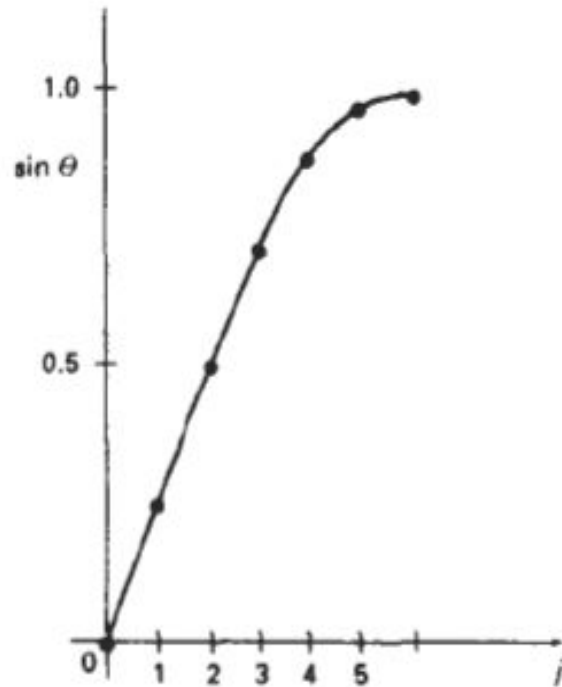
Where Δt the time difference between the two key frames

$j=1,2,\dots,n$

- We can model decreasing speed (deceleration) with sine in the range $0 < \theta < \frac{\Pi}{2}$
- The time position of an in-between is now defined as

$$tB_j = t_1 + \Delta t \sin \frac{j\Pi}{2(n+1)} \quad j=1,2,\dots,n$$

- A plot of this function and the decreasing size of the time intervals for five in-betweens



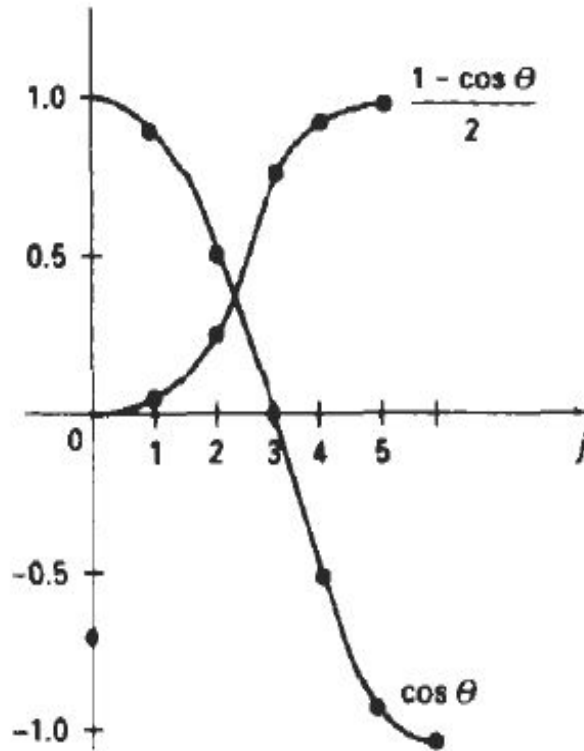
- Often, motions contain both speed-ups and slow-downs. We can model a combination of increasing-decreasing speed by first increasing the in-between time spacing, then we decrease this spacing.
- A function to accomplish these time changes is

$$\frac{1}{2}(1 - \cos \theta)$$

- The time for the j th in-between is now calculated as

$$tB_j = t_1 + \Delta t \left\{ \frac{1 - \cos[j \Pi / (n + 1)]}{2} \right\}$$

- Time intervals for the moving object first increase then the time intervals decrease,



MOTION SPECIFICATIONS

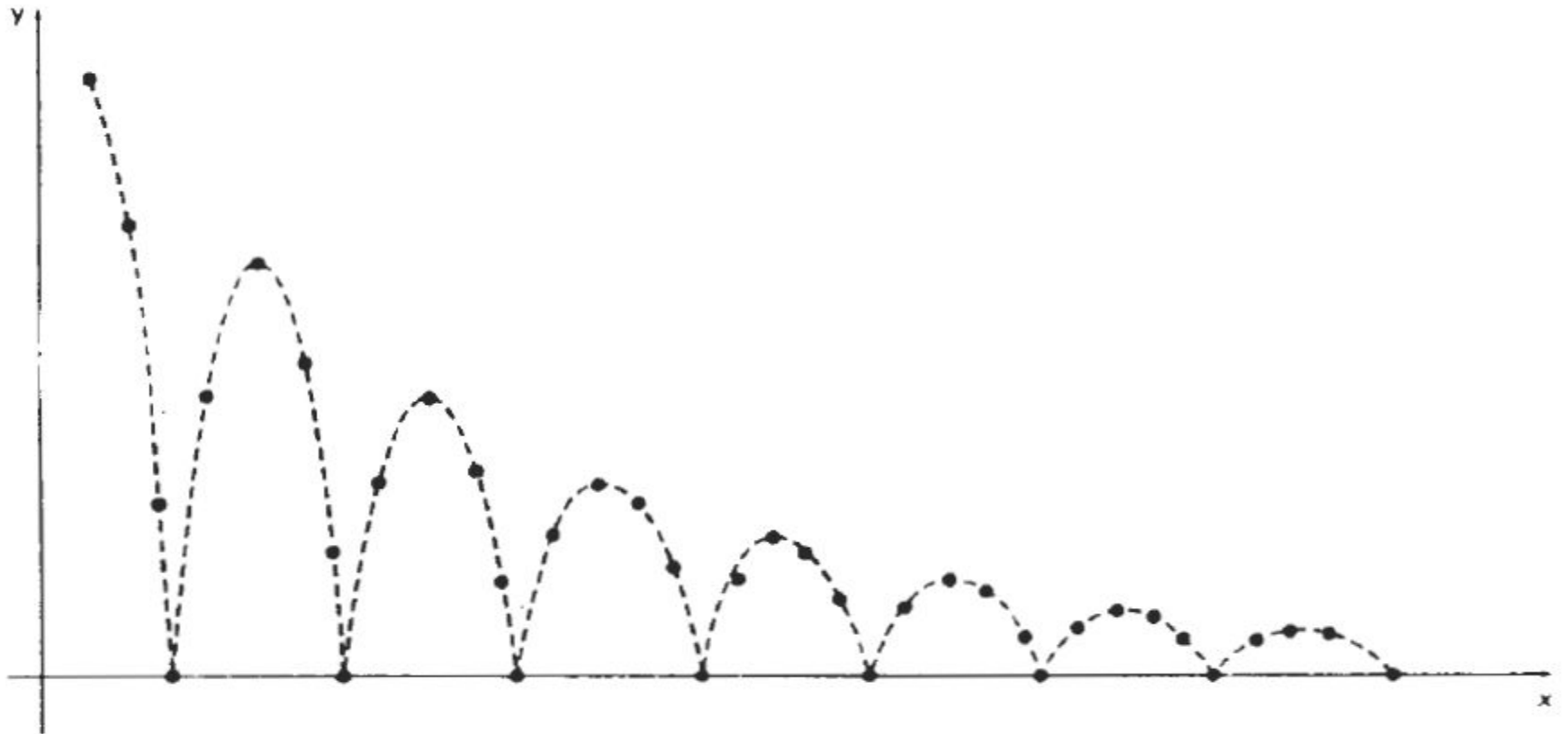
- There are several ways in which the motions of objects can be specified in an animation system
 - Direct Motion Specification
 - Goal-Directed Systems
 - Kinematics and Dynamics

Direct Motion Specification

- Explicitly give the rotation angles and translation vectors are given.
- Then the geometric transformation matrices are applied to transform coordinate positions.
- We could use an approximating equation to specify certain kinds of motions.
- We can approximate the path of a bouncing ball, for instance, with a damped, rectified, *sin* curve

$$y(x) = A | \sin(\omega x + \theta_0) | e^{-kx}$$

- where A is the initial amplitude, ω is the angular frequency, θ , is the phase angle, and k is the damping constant.
- These methods can be used for simple user-programmed animation sequences.



Advantages

- Easily and explicitly parameters are provided to any object.
- Coordinate positions are easily/ applied to transform the object.

Disadvantages

- No implicit parameters can be transform.
- Acceleration of any object is not possible.

Goal-Directed Systems

- We can specify the motions that are to take place in general terms that abstractly describe the actions.
- These systems are referred to as *goal direct* because they determine specific motion parameters given the goals of the animation.
- The input directives are then interpreted in terms of component motions that will accomplish the selected task.
- For example, we could specify that we want an object to "walk" or to "run" to a particular destination. Or we could state that we want an object to "pick up" some other specified object

Kinematics and Dynamics

- We can also construct animation sequences using *kinematic* or dynamic descriptions.
- With a kinematic description, we specify the animation by giving motion parameters (position, velocity, and acceleration) without reference to the forces that cause the motion
- For constant velocity (zero acceleration), we designated motions of rigid bodies in a scene by giving an initial position and velocity vector for each object.

- An alternate approach is to use inverse kinetics. Here, we specify the initial and final positions of objects at specified times and the motion parameters are computed by the system.
- This method is often used with complex objects by giving the positions and orientations of an end node of an object, such as a hand or a foot.
- The system then determines the motion parameters of other nodes to accomplish the desired motion.

- Dynamic descriptions, require the specification of the forces that produce the velocities and accelerations.
- Descriptions of object behavior under the influence of forces are generally referred to as a physically based modeling.
- Object motions are obtained from the force equations describing physical laws, such as Newton's laws of motion for gravitational and friction processes
- use inverse dynamics to obtain the forces, given the initial and final positions of objects and the type of motion.

- Applications of physically based modeling include complex rigid-body systems and such nonrigid systems as cloth and plastic materials.
- Typically, numerical methods are used to obtain the motion parameters incrementally from the dynamical equations using initial conditions or boundary values.

Conventional animation

- Conventional animation is created in a fixed sequence.
- The story for the animation is written, then a **storyboard** is laid out.
- A storyboard is an animation in outline form—a high level sequence of sketches showing the structure and idea of the animation.
- The **soundtrack** is recorded, and the layout is produced and the soundtrack is read.

- Certain **key frames** of the animation are drawn-these are the frames in which the entities being animated are at extreme or characteristics position,
- The intermediate frames are then filled in (inbetweening) and trial film is made(pencil test).
- The pencil test frames are then transferred to cels either by hand copy in ink or by photocopying.

- In multiplane animation multiple cels are used, some for background and some for foreground.
- The cels are colored in or painted, and are assembled into the correct sequence, then they are filmed.
- The people producing the animation have distinct roles
 - Design the sequence
 - Draw key frame
 - Inbetweners
 - Painting final cels

Computer Assisted animation

- By placing several small low resolution frames of an animation in a rectangular array the equivalent of a pencil test can be generated using the pan-zoom feature available in frame buffer.
- The frame buffer can take a particular portion of such an image, move it to the centre of the screen, and then enlarge it to fill the entire screen.
- This process can be repeated on the several frames of the animation stored in the single image.

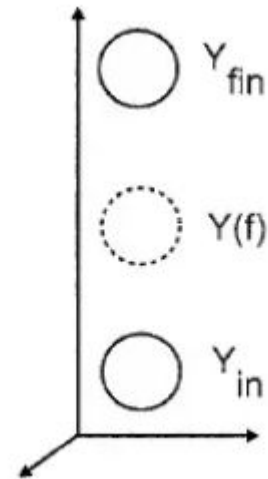
Interpolation

- Computing the in-between frames is done by taking the information in the key frames and averaging it in some way.
- This calculation is called interpolation.
- Interpolation creates a sequence of in-between property values for the in-between frames.
- The type of interpolation used depends on the kind of final motion desired
- Interpolation can be used to calculate different properties of objects in the in-between frames-properties such as the position of objects in space, its size, orientation, color etc

Linear Interpolation

- In linear interpolation, the values of a given property is estimated between two known values on a straight-line basis (hence the term linear).
- In simpler terms, linear interpolation takes the sum of the property values in the two key frames and divides by the number of frames needed to provide as many equally spaced in-between frames as needed.

- *For example, consider a model P , with a position along the y -axis of Y_i , at the initial key frame K_{in} .*
- *At the final key frame, K_{fin} , her position has moved to Y_{fin} .*
- *Now, say we want only one in-between frame f . This frame would be midway between the two key frames, and P would be located mid-way between the positions Y_i , and Y_{fin}*



- *Mathematically, the position $Y(f)$ at Frame f , is given by*

$$Y(f) = Y_{in} + (Y_{fin} - Y_{in}) / 2 = (Y_{fin} + Y_{in}) / 2$$

- Now, suppose we want two in-between frames, $f1$ and $f2$. The model would then be defined such that at fly it occupies a position, $Y(f1)$, which is one third the distance from Y_{in} ; and at frame $f2$, it occupies a position $Y(f2)$, which is two thirds the distance from Y_{in}

Mathematically,

$$Y(f1) = Y_{in} + (Y_{fin} - Y_{in}) / 3$$

$$Y(f2) = Y_{in} + 2 * (Y_{fin} - Y_{in}) / 3$$

- *If we were to extend our discussion to a total of N frames (including the initial and final key frames), we would have $N-2$ in-between frames.*
- *The position Y at any frame f can be calculated by the equation*

$$Y(f) = Y_{in} + f * (Y_{fin} - Y_{in}) / (N - 1)$$

$$0 \leq f \leq N - 1$$

At frame $f = 0$, we attain $Y(0) = Y_{in}$: the initial key frame.

At frame $f = N-1$, $Y(N-1) = Y_{fin}$: the final key frame.

- *We can define similar equations for interpolating the position of P along the x and z-axis*

$$X(f) = X_{in} + f^*(X_{fin} - X_{in}) / (N - 1)$$

$$Z(f) = Z_{in} + f^*(Z_{fin} - Z_{in}) / (N - 1)$$

$$0 \leq f \leq N - 1$$

Non Linear Interpolation

- *For most animation situations, linear interpolation is not an effective animation technique.*
- *More complex non-linear interpolation techniques are employed for simulating effects in nature.*

- *We can interpolate key frame positions of an animation using the nonlinear parametric equations*

$$X(f) = a_x f^3 + b_x f^2 + c_x f + d_x$$

$$Y(f) = a_y f^3 + b_y f^2 + c_y f + d_y$$

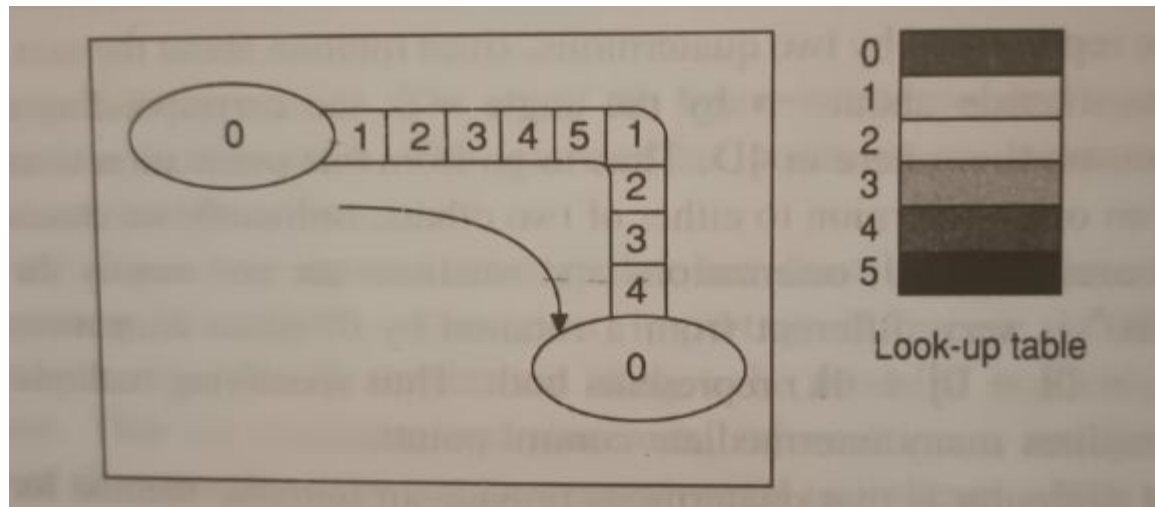
$$Z(f) = a_z f^3 + b_z f^2 + c_z f + d_z$$

$$f = (f - \text{Init}f) / (N - 1)$$

$$\text{Init}f \leq f \leq N \Rightarrow 0 \leq f \leq 1$$

Simple animation effects

- The simplest method is to cycle the colors in the lut(replace the color i with $i-1 \bmod n$), thus changing the colors of the various pieces of the image.



- Each pieces of image is labeled with its lut index. The lut is shown at the right.
- By cycling colors 1 through 5, we can generate an animation of material flowing through the pipe.
- Using this lut animation is a great deal faster than sending an entire new pixmap to its frame buffer for each frame.
- Assuming 8 color bits per pixel in a 640x512 frame buffer, a single image contains 320KB of information.
- New values for the lut can be sent very rapidly. So, lut animation become jerky.

- We can combined lut with pan-zoom movie technique to over come the problem.
- Next method is hardware-based animation technique called sprites.
- A sprite is a small rectangular region of memory that is mixed with the rest of the frame buffer memory at the video level.
- The location of sprite at any time is specified in registers in the frame buffer, altering the values in these registers causes the sprite to move.

- The sprite may hide the frame buffer values at each pixel, or may be blended with them.
- We can use sprites to implement cursors in frame buffers, and also to generate animations by moving the sprite around on top of a background image.
- Some frame buffers have been designed to allow several sprite with different priorities, so that some sprite can be on top of others.
- Use of sprite : gaming

